

field, please see Zendesk documentation.

Reordering user fields

For information on reordering user fields, please see Zendesk documentation.

Deactivating user fields

For information on deactivating a user field, please see Zendesk documentation.

Deleting user fields

{{% alert title="Note" color="warning" %}}

You can only delete a user field if it is deactivated.

{{% /alert %}}

For information on deleting a user field, please see Zendesk documentation.

SECTION: security/customer-support-operations/docs/zendesk/views.md

{{% alert title="Note" color="primary" %}}

This is an informational page for Zendesk views. It may not reflect the way we actually manage Zendesk views

If you are looking for information about managing it, please see views workflow

{{% /alert %}}

What are Zendesk views?

As per Zendesk:

> Views are a way to organize your tickets by grouping them into lists based on certain criteria. For example, you can create a view for unsolved tickets that are assigned to you, a view for new tickets that need to be triaged, or a view for pending tickets that are awaiting response. Using views can help you determine what tickets need attention from you or your team and plan accordingly.

Currently, Zendesk has 3 view types:

- Default: Pre-defined views created by Zendesk
- Shared: Views created by the Zendesk Administrator(s) (ie. Customer Support Operations)
- Personal: Views created by you and usable only by you

Currently, Zendesk views have some limitations:

- Views cannot use criteria that is not "defined", meaning it must be selectable data (text fields will not work, as an example).
- You can only display up to 100 shared views
- You can only display up to 10 personal views
- Views will not include archived tickets (i.e. Closed tickets after 120 days.)

View categorization

For information on view categorization, please see Zendesk documentation.

Creating a view

For information on creating a view, please see Zendesk documentation.

Editing a view

For information on editing a view, please see Zendesk documentation.

Cloning a view

For information on cloning a view, please see Zendesk documentation.

Deactivating a view

For information on deactivating a view, please see Zendesk documentation.

Deleting a view

{{% alert title="Note" color="warning" %}}

You can only delete a view if it is deactivated.

{{% /alert %}}

For information on deleting a view, please see Zendesk documentation.

SECTION: security/customer-support-operations/docs/zendesk/webhooks.md

{{% alert title="Note" color="primary" %}}

This is an informational page for Zendesk webhooks. It may not reflect the way we actually manage Zendesk webhooks/

If you are looking for information about managing it, please see webhooks workflow

{{% /alert %}}

What are Zendesk webhooks

As per Zendesk:

> A webhook sends an HTTP request to a specified URL in response to an event, such as a trigger or automation firing in Zendesk Support. Web developers typically use webhooks to invoke behavior in another system.

Creating a webhook via Zendesk

For information on creating a webhook in Zendesk, please see the Zendesk documentation.

Editing a webhook via Zendesk

To edit a webhook in Zendesk, you first need to go to the Admin Center (Zendesk Global / Zendesk US Federal). From there, you need to go to the Webhooks page (Apps and integrations > Webhooks > Webhooks).

From here, locate the webhook in question, hover over it, and click the 3 dots on the right-hand side of the row. Doing so will have a menu pop up with the Edit option.

Doing so will bring up the webhook editor. You can now edit the webhook, test the changes (see Testing a webhook for more info), and then save your changes by clicking the blue Update button at the bottom-right of the page.

Deleting a webhook via Zendesk

To delete a webhook in Zendesk, you first need to go to the Admin Center (Zendesk Global / Zendesk US Federal). From there, you need to go to the Webhooks page (Apps and integrations > Webhooks > Webhooks).

From here, locate the webhook in question, hover over it, and click the 3 dots on the right-hand side of the row. Doing so will have a menu pop up with the Delete option.

Clicking this will cause a pop-up to appear asking you to confirm the deletion. Click the red Delete webhook button to do so.

Testing a webhook

For information on creating a webhook in Zendesk, please see the Zendesk documentation.

Viewing a webhook's activity

To view a webhook's activity in Zendesk, you first need to go to the Admin Center (Zendesk Global / Zendesk US Federal). From there, you need to go to the Webhooks page (Apps and integrations > Webhooks > Webhooks).

From here, locate the webhook in question, and click on its name. On this page, click the Activity tab to see all the runs of the webhook. You can click on the Invocation ID of the run to get more information on said run (on the side menu that appears, click the timestamp for more details).

SECTION: security/customer-support-operations/docs/zendesk/zd-sfdc-sync.md

{{% alert title="Note" color="danger" %}}

This is an informational page for the Zendesk-Salesforce sync.

If you are looking for information about managing it, please see Zendesk-Salesforce sync workflow

{{% /alert %}}

What is the Zendesk-Salesforce Sync?

The Zendesk-Salesforce Sync (ZD<>SFDC sync for short) is a set of scripts that are run via GitLab CI/CD to handle the interaction between Zendesk and Salesforce. What it does exactly depends on the project itself:

Name	What it does
----- -----	----- -----
salesforce-cases	Creates and updates Salesforce cases based on Zendesk tickets
zd-sfdc-sync-global	Syncs Salesforce account data into Zendesk organizations
zd-sfdc-sync-us-federal	Syncs Salesforce account data into Zendesk organizations
	Syncs Salesforce contacts data into Zendesk users

Zendesk Global organizations sync

This first gathers the data from Salesforce. This is done via the following SOQL (Salesforce Object Query Language) query:

<details>
<summary>SOQL query used</summary>

```
sql
SELECT
  AccountID18c,
  Name,
  Type,
  UltimateParentSalesSegmentEmployeeesc,
  AccountOwnerCalcc,
  TechnicalAccountManagerNameec,
  GSHealthScoreColorc,
  RestrictedAccountc,
  SolutionsArchitectLookupr.Name,
  AccountDemographicsGeoc,
  LatestSoldToContactr.Email,
  LatestSoldToContactr.Name,
  PartnerTrackc,
  PartnersPartnerTypeec,
```

```

SupportHoldc,
AccountRiskLevelc,
SupportInstancec,
(
  SELECT
    Id,
    Name,
    ZuoraProductNamesc,
    ZuoraEffectiveEndDatesc,
    ZuoraQuantityc,
    ZuoraTotalContractValuec,
    SubscriptionStatusc
  FROM ZuoraR00N40000001IGjTEAUr
  WHERE
    ZuoraEffectiveEndDatesc != NULL
),
(
  SELECT
    Name,
    ZuoraSoldToWorkEmailc
  FROM ZuoraR00N40000001kyLcEAlr
  WHERE
    IsDeleted = false
  ORDER BY CreatedDate ASC
  LIMIT 1
)
FROM Account
WHERE
  Type != 'Prospect' AND
  (
    Type IN ('Customer', 'Former Customer') OR
    (
      Type = 'Partner' AND
      PartnersPartnerStatusc IN ('Authorized', 'Former') AND
      PartnersPartnerTypec IN ('Alliance', 'Channel') AND
      PartnerTrackc IN ('Open', 'Select', 'Technology')
    )
  )
)

```

</details>

This data is then processed by the script to verify the account's subscriptions (both Customer and Zuora). It also locates the sold-to emails of the subscriptions to later attempt user creation (if it is a new organization in Zendesk).

While doing this, it also locates accounts that are classified as "greatly expired" (meaning they have been expired for 3 or more years). It puts these into a separate object for use later.

From here, it then gathers all the organization data from Zendesk Global. This does very little actual processing of the data, short of ignoring tags that aren't related to the sync itself.

The scripts then compares the data from Salesforce and the data from Zendesk Global. From this comparison, it locates Zendesk Global organizations that need to be updated and ones that need to be created.

The scripts will then begin syncing this information to Zendesk Global, updating organizations that need updating and creating the ones that need creation. For organizations that need to be created, it will also attempt to create contacts for them based off the information of the subscriptions (gathered earlier).

```
{{% alert title="Note" color="danger" %}}
```

In the event the contact creation fails (or there is no contact), a message will be posted in the supportoperations slack channel. We act on these to attempt manual association wherever possible.

```
{{% /alert %}}
```

Zendesk US Government organizations sync

This first gathers the data from Salesforce. This is done via the following SOQL (Salesforce Object Query Language) query:

```
<details>
<summary>SOQL query used</summary>
```

```
sql
SELECT
  AccountID18c,
  Name,
  UltimateParentSalesSegmentEmployeeesc,
  AccountOwnerCalcc,
  TechnicalAccountManagerNameec,
  GSHealthScoreColorc,
  RestrictedAccountc,
  SolutionsArchitectLookupr.Name,
  SupportHoldc,
  SupportInstancec,
  (
    SELECT
      Id,
      Name,
      ZuoraProductNameec,
      ZuoraEffectiveEndDateec,
      ZuoraQuantityc,
      ZuoraTotalContractValueec,
      SubscriptionStatusc
    FROM ZuoraR00N40000001IGjTEAUr
```

```

)
FROM Account
WHERE
  Id IN
  (
    SELECT
      ZuoraAccountc
    FROM ZuoraSubscriptionProductChargec
    WHERE
      SubscriptionStatusc = 'Active' AND
      ZuoraEffectiveEndDate >= {(Date.today - 91.days).iso8601} AND
      (
        Name IN (
          '12x5 US Citizen Support - 1 Year',
          '12x5 US Citizen Support - 2 Year',
          '12x5 US Citizen Support - 3 Year',
          '12x5 US Citizen Support - Monthly',
          '24x7 US Citizen Support - 1 Year',
          '24x7 US Citizen Support - 2 Year',
          '24x7 US Citizen Support - 3 Year',
          '24x7 US Citizen Support - Monthly',
          'GitLab Dedicated for US Public Sector - Ultimate - 1 Year',
          'GitLab Dedicated for US Public Sector - Ultimate - 2 Year',
          'GitLab Dedicated for US Public Sector - Ultimate - 3 Year',
          'GitLab Dedicated for US Public Sector - Ultimate - Monthly'
        ) OR
        ZuoraAccountr.SupportInstancec = 'federal-support'
      )
  )
)

```

</details>

This data is then processed by the script to verify the account's subscriptions (both Customer and Zuora).

From here, it then gathers all the organization data from Zendesk US Government.

The scripts then compares the data from Salesforce and the data from Zendesk US Government. From this comparison, it locates Zendesk US Government organizations that need to be updated and ones that need to be created.

The scripts will then begin syncing this information to Zendesk US Government, updating organizations that need updating and creating the ones that need creation.

Zendesk US Government users sync

This first gathers the data from Salesforce. This is done via the following SOQL (Salesforce Object Query Language) query:

<details>

<summary>SOQL query used</summary>

sql

```
SELECT
  Name,
  Email,
  Account.AccountID18c,
  Account.Name
FROM Contact
WHERE
  InactiveContactc = false AND
  Name != " AND
  Email != " AND
  Rolec INCLUDES ('Gitlab Admin') AND
  (
    NOT Email LIKE '%gitlab.com'
  ) AND
  Account.Type IN ('Customer', 'Former Customer') AND
  AccountId IN (
    SELECT
      ZuoraAccountc
    FROM ZuoraSubscriptionProductChargec
    WHERE
      SubscriptionStatusc = 'Active' AND
      ZuoraEffectiveEndDate >= {(Date.today - 91.days).iso8601} AND
      (
        Name IN (
          '12x5 US Citizen Support - 1 Year',
          '12x5 US Citizen Support - 2 Year',
          '12x5 US Citizen Support - 3 Year',
          '12x5 US Citizen Support - Monthly',
          '24x7 US Citizen Support - 1 Year',
          '24x7 US Citizen Support - 2 Year',
          '24x7 US Citizen Support - 3 Year',
          '24x7 US Citizen Support - Monthly',
          'GitLab Dedicated for US Public Sector - Ultimate - 1 Year',
          'GitLab Dedicated for US Public Sector - Ultimate - 2 Year',
          'GitLab Dedicated for US Public Sector - Ultimate - 3 Year',
          'GitLab Dedicated for US Public Sector - Ultimate - Monthly'
        ) OR
        ZuoraAccountr.SupportInstancec = 'federal-support'
      )
  )
)
```

</details>

This data is then processed to remove any contacts with duplicate emails or missing data.

From here, it then gathers all the user data from Zendesk US Government.

The scripts then compare the data from Salesforce and the data from Zendesk US Government. From this comparison, it locates Zendesk US Government users that need to be updated and ones that need to be created. It will use the organization data from the Zendesk US Government organizations sync to determine the organization ID.

The scripts will then begin syncing this information to Zendesk US Government, updating users that need updating and creating the ones that need creation.

Zendesk Salesforce cases sync

This is runs at two periods of time:

- whenever a ticket with an organization associated is created
- whenever a ticket with a matching Salesforce case is closed

For when a ticket is created, it will create a Salesforce case based off the Zendesk Global ticket data.

For when a ticket is closed, it will update the corresponding case to indicate it has been closed.

SECTION: security/customer-support-operations/workflows/_index.md

SECTION: security/customer-support-operations/workflows/annual-review.md

Once a year, in December, we review all unused items to determine if they need to be kept around. This is done to keep our systems and repos clean.

Zendesk

Articles

If you see archived items older than 6 months, you are good to delete them (this should be done directly in Zendesk).

Automations

If you see deactivated items older than 6 months, you are good to delete them. Remember you need to delete them from the repo and from Zendesk itself.

Macros

We do two levels of review here, one for long-standing deactivated items and one for unused macros.

Long-standing deactivated macros

If you see deactivated items older than 6 months, you are good to delete them. Remember you need to delete them from the repo and from Zendesk itself.

Unused macros

This is a bit more involved and takes more time. You will start by generating a list of macros that have not been used in 30 days (See macro usage information).

With that list, you will create an issue in the correct tracker:

- For Zendesk Global: support-team-meta
- For Zendesk US Government: us-government-support-meta

The issue should use the following information:

- Title: xxxx
- Description:

plaintext
Annual Unused Macro review

Greetings Support! We are doing our annual review of our Zendesk instance!

Part of this is reviewing unused macros. The following macros have not been used in the last 30 days:

- [x] LIST
- [x] OF
- [x] ITEMS

Please review the above list and uncheck the box of macros you wish to keep in place. All checked boxes after the due date will be deactivated.

/due in two weeks

/assign me

/label ~"request-for-comments" ~"FY26 OKR::Not Applicable"

Make sure to replace the list with the list of actual macro names (and that the boxes are checked by default).

After creation, make sure to create a SWIR item for this issue.

After two weeks, take the list of checked boxes and go deactivate those macros (see Macros workflow).

Organization Fields

If you see deactivated items older than 6 months, you are good to delete them. Remember you need to delete them from the repo and from Zendesk itself.

Ticket Fields

If you see deactivated items older than 6 months, you are good to delete them. Remember you need to delete them from the repo and from Zendesk itself.

Ticket Forms

If you see deactivated items older than 6 months, you are good to delete them. Remember you need to delete them from the repo and from Zendesk itself.

Triggers

If you see deactivated items older than 6 months, you are good to delete them. Remember you need to delete them from the repo and from Zendesk itself.

User Fields

If you see deactivated items older than 6 months, you are good to delete them. Remember you need to delete them from the repo and from Zendesk itself.

Views

If you see deactivated items older than 6 months, you are good to delete them. Remember you need to delete them from the repo and from Zendesk itself.

SECTION: security/customer-support-operations/workflows/audits.md

What are audits

Audits are what we call the process (and core responsibility) that involves reviewing who has what access to various platforms.

Why do we perform audits

There are a good number of reasons that could be made for doing audits, but the biggest ones for us are:

- Ensure no security issues are occurring in the agent/user space.
- Ensure we have accurate information for procurements and renewals.
- Ensure we follow best practices for reviewing who is using the various systems we manage.

How to perform an audit

On the 1st day of each quarter, issues will be generated via our System Audits project. The schedule pipeline there will do many checks and verifications for us and generate issues with the reports of what it finds. The ones the issues are assigned to are responsible for reviewing the found results and acting on any items that need to be rectified.

What is checked for each audit

- Calendly
 - Number of people with Owner role is 1
 - All members of Customer Support Operations are admins
 - No one outside of Customer Support Operations are admins
 - All Support team members have accounts
 - The calendly URL in the support-team.yaml is correct for each person
 - No one outside of Support team members have accounts
- GitLab
 - Memberships for gitlab-support-readiness/all-readiness
 - Memberships for gitlab-support-readiness/all-developers
 - Memberships for gitlab-support-readiness/all-reporters
 - Memberships for gitlab-support-readiness/readiness-us-gov-only
 - Memberships for gitlab-support-readiness/developers-us-gov-only
 - Memberships for gitlab-support-readiness/reporters-us-gov-only
- Zendesk Global
 - All Support team members have accounts
 - Report all full agents outside of Support team members that have accounts
 - The Zendesk data in the support-team.yaml is correct for each person
 - Checks for any agents in the suspended state
- Zendesk US Government
 - All Support team members with US Government focus have accounts
 - Report all full agents outside of Support team members with US Government focus that have accounts
 - The Zendesk data in the support-team.yaml is correct for each person
 - Checks for any agents in the suspended state

SECTION: security/customer-support-operations/workflows/change-management.md

{{% alert title="Note" color="danger" %}}

The exact process for change management can vary from project to project and item to item, so when in doubt, it is best to refer to the specific workflow for the item in question.

{{% /alert %}}

Types of deployments

Standard deployments

A standard deployment for us happens on the first of every month. We center our milestones around these. We utilize scheduled pipelines for these items so everything is automated. Changes in the repos following standard deployments are all synced to the systems via these on the deployment date.

Ad-hoc deployments

For some items, it makes little sense to deploy them on specific dates. For these items, they are deployed when the changes are made to the repos (or in the system itself).

Exception deployments

There are situations where an item that normally uses a standard deployment needs to be done before the actual deployment date it would normally fall into.

When these situations occur, the general process for them should go as follows:

1. The requester asks for an exception deployment

1. @dtragjasi and/or @jcolyer will make a comment detailing what the impacts of the exception deployment will be.

- This is done by reviewing what has been merged and is "queued for deployment" in the various areas the topic entails.

1. The Sr. Support Engineering Manager - Operations will review the request and impacts to make a decision on the feasibility and acceptance of the request. He will present the consequences, impact, etc. to the requester.

1. If the requester agrees with the Sr. Support Engineering Manager - Operations statement, they will confirm they still wish to proceed.

Should all parties approve the request, we will then perform an exception deployment. The exact means of doing so can vary, so please refer to the specific workflow for the item(s) in question.

Special Deployments

This is a categorization for areas that require very specific deployment methods that fall outside of Standard, Exception, and Ad hoc deployments. For more information on these, see the workflow page of the item itself.

What uses what type of deployment

To determine the type of deployment something uses, please refer to that item's workflow page.

SECTION: security/customer-support-operations/workflows/cmps.md

Eligibility Criteria

To be eligible for a contact management project, all of the following criteria must be met:

- The requester must be associated to an organization
- The organization must have an active subscription with GitLab (it cannot be expired or within the grace period)
- The organization must have a qualifying subscription, which would be one of the following:
 - A GitLab Premium subscription (Self-Managed or gitlab.com)
 - A GitLab Ultimate subscription (Self-Managed or gitlab.com)
 - A GitLab Dedicated subscription
- The organization's valid subscription must be for at least 50 seats
- The requester must be one of the following:
 - The sold-to (purchasing party) of the subscription
 - An owner on the gitlab.com namespace

If any of those criteria are not met, you should reply to the ticket with the Support::Support-Ops::Not qualified for CMP macro.

If they have met all criteria, you can proceed to the setup section below.

Setup

The setup and creation of a contact management project is completely performed via the Zendesk Global app Support Ops Super App. As such, you should use that to kick the process off after you have done the needed checks. The app will ask you for a comma-separated list of emails to use for the project's developers. After submission, a combination of the app and the ticket processor will handle the rest.

Manual setup

{{% alert title="Note" color="danger" %}}

The below information details how to manually setup a contact management project. This should only ever be used in the event of a unfixable error from the application. Consider contacting the Fullstack Engineer, Customer Support Operations before proceeding in doing this manually.

{{% /alert %}}

These are setup via several stages that all work together to reflect a complete contact management project.

Intro stage

In this stage, a customer has requested to have a contact management project setup via a Support Ops ticket. Before we can begin the setup, we need to confirm with the requester which user(s) will be the ones able to edit the contacts.yaml file. To do this, use the macro Support::Support Ops::Contact management intro.

Once the requester has confirmed the user(s) who will be maintaining the contacts.yaml file, you need to ensure the users provided are associated to the organization currently. If they are not, you should associate them to the organization first.

After doing so, you will need to get a list of the organization's current contacts for use in

the setup stage.

After doing so, you are good to move to the next stage.

Jason Pro-tip

I would recommend exporting them as a CSV file via Advanced search. When doing so, make sure the only columns that display are the user's name and email.

With the CSV file downloaded, you could then run the following code via terminal to change the CSV format into the YAML format you will need later:

```
bash
awk -F',' '{if ($1!="Query"&&$1!=""&&$1!="name\\") {print "- name: \"$1\"\\n email: \"$2\"}}'
csvfile
```

This will work the majority of the time and make it a bit easier down the line.

Project creation

For organizations without an existing collaboration project, we create all contact management projects within the zd-global-orgs group.

1. Navigate to that group and click the blue New project button at the upper right part of the page.
1. Select Create from template
1. Click the Group tab
1. Click the blue Use template button to the right of Contact Management Project Template
1. Set the Project name to Organization XXXX (replacing XXXX with the Zendesk Organization ID)
 - Ensure the Project slug is organization-xxxx (replacing xxxx with the Zendesk Organization ID)
1. Click the blue Create project button

After creation the project, you need to double check all settings are accurate. In theory, the template project will handle that, but it's always best to be sure.

The items to check are:

- Settings
 - Settings > General
 - Visibility, project features, permissions
 - Project visibility: Private
 - Enable CVE ID requests in the issue sidebar: disabled
 - Forks: disabled
 - Git Large File Storage (LFS): disabled
 - CI/CD: disabled
 - Container registry: disabled
 - Analytics: disabled
 - Requirements: disabled

- Security & Compliance: disabled
- Wiki: disabled
- Snippets: disabled
- Package registry: disabled
- Pages: disabled
- Monitor: disabled
- Environments: disabled
- Feature flags: disabled
- Infrastructure: disabled
- Releases: disabled
- Settings > Repository
 - Branch defaults
 - Default branch: master
 - Protected branches
 - Branch: master
 - Allowed to merge: 2 roles (Maintainers+Developers)
 - Allowed to push: 2 roles (Maintainers+Developers)
 - Allowed to force push: disabled
 - Code owner approval: disabled
- Project files
 - CONTACTS.md
 - Verify the contents match what is detailed here
 - contacts.yaml
 - Verify the contents match what is detailed here

After everything looks good to go, make sure to copy the project's ID. You'll need it in the next step.

Webhook setup

Next, you'll need to setup the webhook for the project. To do this, navigate to Settings > Webhooks on the project.

You then need to create a new webhook using the following information:

- URL: `https://gitlab.com/api/v4/projects/39250741/ref/master/trigger/pipeline?token=TOKEN&variables[PROJECTID]=PROJECTID`
 - Replace TOKEN with the Contact Management Project Token found within the Customer Support Operations Vault in 1Password.
 - Replace PROJECTID with the project's ID.
- Click the bubble next to Mask portions of URL
 - Put the token value in the text field for Sensitive portion of URL
 - Put TOKEN in the text field for How it looks in the UI
 - Verify the URL preview value now looks like:
 - `https://gitlab.com/api/v4/projects/39250741/ref/master/trigger/pipeline?token={TOKEN}&variables[PROJECTID]=123456789`
- Check the box next to Push events
 - Select Wildcard pattern and enter master in the text field

- Check the box next to Enable SSL verification

Once all that is in place, click the blue Add webhook button.

Organization update

Next, you will need to update the organization within Zendesk:

- You want to set the Contact Management Project ID field to have the project ID.

Files setup

Next, you will add the contact's and add the organization ID to the contacts.yaml file of the project.

Every contact within that file uses the format:

yaml

- name: NAMEOFCONTACT
email: EMAILOFCONTACT

Where NAMEOFCONTACT is the contact's name and EMAILOFCONTACT is the contact's email. Both are required, so if you do not know the NAMEOFCONTACT, use the EMAILOFCONTACT value.

An example of a working and valid contacts.yaml can be found [here](#).

You should ensure the contacts listed matching what the Zendesk organization currently has in place.

Once that is all setup, commit the changes to the file (this will kick off a sync pipeline).

User invites

```
{{% alert title="Note" color="danger" %}}
```

Be exceedingly careful here. You can cause a compromise if you invite users at the wrong permission level.

```
{{% /alert %}}
```

Next, you need to invite the user(s) who will maintain the contacts.yaml file. You should have a list of who to invite from your previous work in the ticket.

To invite the users, do the following:

1. Select Manage > Members from the menu in the left hand side of the page
1. Click the blue Invite members button at the top-right hand side of the page
1. Enter the email(s) to use under the Username or email address text box
1. Under Select a role, select Developer

1. Click the blue Invite button on the bottom-right of the modal

Final stage

With all of that done, the final part is to update the ticket using the macro Support::Support Ops::Contact management project setup.

From there, the user might come back with questions or a need for assistance. Do your best to guide them and assist them however you can. If you get stuck, reach out in the support operations slack channel.

Removal

```
{{% alert title="Note" color="warning" %}}
```

If the customer is over 30 contacts at the time of the removal request, ask the customer to use the contact management project to lower their contact numbers down to 30 first (as that is the limit for organization contacts when not using a contact management sync project).

```
{{% /alert %}}
```

To remove the setup, you need to do two steps:

1. Delete the contact management project
1. Remove the value of the Contact Management Project ID on the organization

Common troubleshooting

Pipeline failed due to "No org found"

This occurs when the pipeline was run before the Organization in Zendesk has the Contact Management Project ID field populated. Due to caching, this error will persist for up to an hour in normal situations, which is not ideal.

To fix this issue, go to the contact management project itself, go to the webhooks page, click Test, and select Push events. This will cause a new pipeline to run that should work outside of the caching.

Pipeline failed due to "Too many orgs"

This means multiple organizations within Zendesk are using the same project ID. You will need to determine which organization incorrectly is also using the project ID and remove it.

After doing so, you should manually have the webhook trigger by going to the webhooks page, click Test, and select Push events. This will cause a new pipeline to run that should work outside of the caching.

Pipeline failed due to "Error reading YAML file"

This means something within the contacts.yaml file of the project is in a bad formatting. To

rectify this, you will need to do one of the following:

- Create a ticket to the customer asking them to fix it
- Go to the project and fix it yourself

Either method will work in the end, but the latter is quicker and better in terms of customer experience.

If this happens repeatedly to the same customer, consider reaching out to their Account Manager regarding it.

Pipeline failed due to "Invalid project ID"

This means the PROJECTID value sent by the webhook is not a valid value. You need to go review the webhook you setup and correct it to use the correct value.

Pipeline failed due to "Failed to sync contacts"

This means the Zendesk API returned a non-200 status code when it tried to add users to the organization. You will need to manually troubleshoot why this is and rectify the blocking issue.

Pipeline failed due to "Failed to remove users"

This means the Zendesk API returned a non-200 status code when it tried to remove users to the organization. You will need to manually troubleshoot why this is and rectify the blocking issue.

Pipeline failed due to "NoMethodError"

This means something within the contacts.yaml file of the project is missing required information. This can either be because it was left blank or because of a formatting issue (that does not make it an invalid YAML format). To rectify this, you will need to do one of the following:

- Create a ticket to the customer asking them to fix it
- Go to the project and fix it yourself

Either method will work in the end, but the latter is quicker and better in terms of customer experience.

If this happens repeatedly to the same customer, consider reaching out to their Account Manager regarding it.

Pipeline failed due to "Unable to create user"

This reflects a Zendesk caching issue. It normally is a problem indicating that the user was created but the response from the creation task didn't render properly via the API. You can normally wait 5 or so minutes and then re-run the pipeline to resolve this problem. If you continue seeing issues:

- Create the user manually
- Associate the user to the org manually
- Wait 5 minutes
- Run a new sync by triggering the webhook on the project

SECTION: security/customer-support-operations/workflows/contacting-third-parties.md

Several of the systems we manage are made by third-parties. Sometimes, we encounter issues specific to these the third-party applications that cannot be fully resolved by internally. In such scenarios, the Customer Support Operations team will need to contact the application's support system.

Process

1. The agent or a Customer Support Operations team member will create an issue in the Support Ops Project.
1. A Customer Support Operations team member will act as the DRI.
1. The DRI will then triage and investigate the issue at hand, which may include collecting items such as HAR files, Browser versions, and plugins.
1. The DRI will open a ticket with the third-party app's support system, providing detailed information.
 - Always make sure to CC other Customer Support Operations team members
1. The DRI will update all the responses received from the third-party app's support team in the issue till the issue is fixed.

SECTION: security/customer-support-operations/workflows/handover-procedures.md

Customer Support Operations Handover Procedures

This document outlines the handover procedures between different regional teams in the Customer Support Operations group.

Handover Schedule

Support coverage transitions between regions according to the following schedule:

APAC to EMEA Handover

- Time: 06:00 UTC / 14:00 Manila / 16:00 Melbourne / 07:00 Netherlands

EMEA to AMER Handover

- Time: 13:00 UTC / 15:00 Netherlands / 08:00 Houston

AMER to APAC Handover

- Time: 21:00 UTC / 16:00 Houston / 08:00 Melbourne (next day)

Handover Process

Preparing for Handover

1. Review Outstanding Tickets

- Identify tickets that require immediate attention
- Document the status of any critical issues
- Prioritize tickets that should be addressed by the incoming team

2. Update Ticket Documentation

- Ensure all tickets to be handed over have up-to-date notes
- Document any troubleshooting steps already taken
- Note any pending customer communications

3. Review Audit Events channel

- Document any ongoing concerns
- Note any incidents that may impact runners or other job processing

Handover format

The following format is suggested:

markdown

:ticket: TICKET/S NEEDING ASSISTANCE:

-

:redcircle: OTHER ITEM/S NEEDING ASSISTANCE, ATTENTION OR FOLLOW-UP

-

After Handover

1. Transition Period

- Outgoing team remains available after handover for any questions
- Ensure incoming team acknowledges receipt of all critical information by looking for an emoji reaction

SECTION: security/customer-support-operations/workflows/incidents.md

Whenever an incident occurs, you should follow this process.

Create an issue

Always create an issue using the Incident template. The information you put in it initially does not need to be "complete". It will often be a simplified title and the error/problem itself that is occurring.

Create an incident

After the issue has been created, you need to publish an incident via incident.io (so it shows on our status page). To do this:

1. Navigate to Status pages
1. Click on the status page you want to make an incident on
1. Click Publish incident at the top-right
1. Fill out a meaningful Name
1. Set the Status of the incident
 - Investigating: Report an incident
 - This is normally what you would use as the starting point
 - Identified: Problem has been determined and a fix is being made
 - Monitoring: Fix is implementing and we are monitoring the situation
 - Resolved: Everything is good to go
1. Set a meaningful Message for the incident
 - You should include a link to your incident issue here
1. Set the level of impact on Affected components (the value needed depends on the impact of the incident)
 - No impact: The incident does not impact this component
 - Degraded performance: The component is working but at lower than standard performance levels
 - Partial outage: Significant parts of the component is not working
 - Full outage: The component is hard down
1. Click Review incident
1. Review all information for accuracy
1. Click Publish incident

Determine criticality level

After issue creation, you need to determine the criticality level of the incident. This is done by locating the impacted items on the Customer Support Operations System Criticality Google sheet (internal access only).

In the event an incident is impacting multiple systems, use the highest criticality value.

Work to resolve the problem

With that out of the way, you will work to resolve the problem. Make sure to make ample comments on the issue as you do.

As you do, remember to provide periodic updates to the incident.

Updating an incident

To do this:

1. Navigate to Status pages
1. Click on the status page you want to make an incident on
1. Click Publish incident at the top-right
1. Fill out a meaningful Name
1. Set the Status of the incident

- Investigating: Report an incident
 - Identified: Problem has been determined and a fix is being made
 - Monitoring: Fix is implementing and we are monitoring the situation
 - Resolved: Everything is good to go
1. Set a meaningful Message for the incident
 1. Set the level of impact on Affected components
 - No impact: The incident does not impact this component
 - Degraded performance: The component is working but at lower than standard performance levels
 - Partial outage: Significant parts of the component is not working
 - Full outage: The component is hard down
 1. Click Review incident
 1. Review all information for accuracy
 1. Click Publish incident

Escalate

In some situations, you will not be able to resolve an incident quickly enough and will need to escalate it. The time between escalation levels is going to depend on the criticality level of the incident:

Criticality level	Time to Resolve	Escalate up to	Time to escalate to next level
Special notes	-----	-----	-----
Mission Critical	1-2 hours	Level 4	10 minutes without a resolution path
Business Critical	2-4 hours	Level 3	30 minutes without a resolution path
Business Operational	24-48 hours	Level 2	8 hours without a resolution path Move to level 3 is not resolved in time
Administrative	48-72 hours	Level 2	24 hours without a resolution path Move to level 3 is not resolved in time

As an example, if you are working an Administrative level incident:

- You have 24 hours to try to resolve it yourself
- After 24 hours with no clear path to resolution, you must escalate to level 1
- After 48 hours with no clear path to resolution, you must escalate to level 2
- After 72 hours with no clear path to resolution, you must escalate to level 3. This is because it has now surpassed the documented time to resolve the incident

As another example, if you are working a Mission Critical level incident:

- You have 10 minutes to try to resolve it yourself
- After 10 minutes with no clear path to resolution, you must escalate to level 1
- After 20 minutes with no clear path to resolution, you must escalate to level 2
- After 30 minutes with no clear path to resolution, you must escalate to level 3
- After 40 minutes with no clear path to resolution, you must escalate to level 4

Escalation levels

Level	Action
1	Post in team channel asking for assistance
2	Page Customer Support Operations Specialist oncall
3	Page Fullstack Engineer, Customer Support Operations oncall
4	Page Sr. Support Engineering Manager - Operations

Resolve the issue

Once you have resolved the incident, you need to resolve the issue you created. This is normally done by cross-linking MRs to the issue or by detailing what was done to fix the problem.

Once that is done, you can close the issue, unless an After Incident Review is required.

Resolve the incident

See Updating an incident. Remember the Status to use is Resolved.

After Incident Review

{{% alert title="Note" color="danger" %}}

For critical level 1 and 2 incidents, as well as incidents that required an escalation to level 3, an after incident review is required.

{{% /alert %}}

For this, you will utilize the Customer Support Operations After Incident Reviews Google doc (internal only).

Make a duplicate of the Template tab, and then fill it in completely. You can use previous documents as an example of what is needed.

Make sure to notify the Sr. Support Engineering Manager - Operations of the document once you have completely filled it in.

SECTION: security/customer-support-operations/workflows/pagerduty.md

{{% alert title="Note" color="primary" %}}

Pagerduty changes are classified as ad-hoc. Once changes are made in Pagerduty, they are live.

This can mean changes can cause severe problems if you have not thoroughly tested them.

{{% /alert %}}

Making changes

Currently, changes to Pagerduty are made in Pagerduty itself.

SECTION: security/customer-support-operations/workflows/personal-data-requests.md

As we maintain Zendesk, when an Personal Data Requests involves the deletion of an account, we will be notified by the team member working the issue.

When this occurs, you will need to delete the user's account in Zendesk. For information on doing that, please see Deleting End-users

There is no need to "permanently delete" the user, as we have automated processes in place to do this for you.

After you have done so, ensure you update the issue in question to indicate this has been done.

SECTION: security/customer-support-operations/workflows/support_operations-on-call.md

Introduction

The Support Readiness team maintains an on-call rotation for those members focused on Support Operations. This rotation ensures that the support team is always able to deliver support.

Expectations for Support Ops On-call

As part of GitLab Support on-call, Support Readiness members focused on Operations serve in a rotation. The Support Ops on-call is responsible generally for:

1. Address operational emergencies that affect GitLab's ability to deliver support.
1. Work with Support Managers to develop and communicate interim plans to provide continuity in service delivery while root cause analysis is in progress.
1. Identify and address root causes

Manually triggering a PagerDuty notification

You can manually trigger a PagerDuty notification through PagerDuty itself.

Login to gitlab.pagerduty.com and select + New Incident from the upper right corner. Then fill out the form as follows:

- Impacted Service: Customer Support
- Assign to: Support Readiness
- Title: Add a description here

No other fields need to be filled out, therefore you may then click Create Incident

SECTION: security/customer-support-operations/workflows/token-rotation.md

This page documents what to do when a token needs to be rotated (meaning one already existed).

Rotating tokens

For Zendesk API tokens

1. Navigate to the admin panel for the Zendesk instance in question:
 - Zendesk Global
 - Zendesk Global Sandbox
 - Zendesk US Government
 - Zendesk US Government Sandbox
1. Go to the Zendesk API option on the sidebar (under Apps and integrations > APIs)
1. Locate the existing entry for the token and delete it (click on it then click Delete)
1. Click the Add API token button at the top-right of the page
1. Enter an appropriate name (normally the link to the project using it)
1. Copy the API token generated
1. Put the token into place where needed

For gitlab.com personal access tokens

1. Login to the gitlab.com user the token will be created from
1. Navigate to the personal access tokens page
1. Sort through the list of existing tokens to locate the one you need to rotate
1. Click the swirling arrows (if you hover over it, it says Rotate) next to the correct token
1. Copy the API token generated
1. Put the token into place where needed

For gitlab.com project access tokens

1. Navigate to the project itself
1. Go to the Access tokens page (under Settings)
1. Sort through the list of existing tokens to locate the one you need to rotate
1. Click the swirling arrows (if you hover over it, it says Rotate) next to the correct token
1. Copy the API token generated
1. Put the token into place where needed

For gitlab.com pipeline trigger tokens (as a regular user)

1. Login to the gitlab.com user the token will be created by
1. Navigate to the project itself
1. Go to the CI/CD page (under Settings)
1. Expand the Pipeline trigger tokens section
1. Sort through the list of existing tokens to locate the one you need to rotate
1. Delete the existing token
1. Click the Add new token button at the top-right of the section
1. Enter an appropriate name:
 - If a Zendesk webhook, put the link to the webhook itself

- If a Zendesk app, use the format INSTANCE - NAMEOFAPP
 - INSTANCE is the Zendesk instance itself (ex: Zendesk Global, Zendesk US Government)
 - NAMEOFAPP is the name of the app as Zendesk display it
 - If for a CI/CD job within the same project, put the name of the job
 - If for another project, put the link to the project
1. Copy the API token generated
 1. Put the token into place where needed

For gitlab.com pipeline trigger tokens (as a service bot)

1. Create a project access token for the project in question
1. Make note of the project's ID number
1. Use that API token to create a pipeline trigger token via the gitlab.com API

```
bash
curl --request POST \
  --header "PRIVATE-TOKEN: TOKENYOURCOPIED" \
  --form description="APPROPRIATEDESCRIPTIONHERE" \
  "https://gitlab.com/api/v4/projects/PROJECTID/triggers"
```

- TOKENYOURCOPIED is the project access token you copied
 - APPROPRIATEDESCRIPTIONHERE is an appropriate description:
 - If a Zendesk webhook, put the link to the webhook itself
 - If a Zendesk app, use the format INSTANCE - NAMEOFAPP
 - INSTANCE is the Zendesk instance itself (ex: Zendesk Global, Zendesk US Government)
 - NAMEOFAPP is the name of the app as Zendesk display it
 - If for a CI/CD job within the same project, put the name of the job
 - If for another project, put the link to the project
1. Copy the API token generated
 1. Put the token into place where needed

Applying the new token

For Zendesk apps

1. Navigate to the admin panel for the Zendesk instance in question:
 - Zendesk Global
 - Zendesk Global Sandbox
 - Zendesk US Government
 - Zendesk US Government Sandbox
1. Go to the Zendesk Support apps option on the sidebar (under Apps and integrations > Apps)
1. Locate the app in question and click on it
1. Locate the field that needs the API token in question
1. Put the token into that field
 - NOTE Do not populate or edit any other fields
1. Click the blue Update button at the bottom of the page

For Zendesk webhooks

1. Navigate to the admin panel for the Zendesk instance in question:
 - Zendesk Global
 - Zendesk Global Sandbox
 - Zendesk US Government
 - Zendesk US Government Sandbox
1. Go to the Webhooks option on the sidebar (under Apps and integrations > Actions and webhooks)
1. Locate the webhook in question
1. Click the 3 vertical dots to the far-right of the webhook in question
1. Click the Edit option
1. Click the Next link at the bottom-right of the page
1. Replace the old token with the new token where needed
1. Click the blue Update button at the bottom-right of the page

For gitlab.com CI/CD variables

1. Navigate to the project itself
1. Go to the CI/CD page (under Settings)
1. Expand the Variables section
1. Sort through the list of existing variables to locate the one you need to replace
1. Click the pencil icon at the far-right of the variable (if you hover over it, it says Edit)
1. Put the new token in the Value field
1. Click the blue Save changes button

For gitlab.com webhooks

Due to being unable to edit the value of masked sections in webhooks, we have to "delete and create" it to rotate a token

1. Navigate to the project itself
1. Go to the Webhooks page (under Settings)
1. Locate the webhook in question and copy all relevant information from it (the URL, what it triggers on, etc.)
1. Delete the webhook in question
1. Re-create the webhook using the relevant information and the new token

OAuth Integrations

Integrating a new OAuth Application into Zendesk

Adding an OAuth integration requires Owner access to Zendesk.

After an access request is approved:

1. Remove the Okta login requirement for the integration user
2. Log in as the integration user
3. Perform the OAuth flow as directed by the application.
 - Verify the scopes requested are documented and approved in the access request. If they are not, STOP.
4. Log out as the integration user

5. Restore Okta login requirements for the integration user.

SECTION: security/customer-support-operations/workflows/workato.md

{{% alert title="Note" color="primary" %}}

Workato changes are classified as ad-hoc. Once changes are made in Workato, they are live.

This can mean changes can cause severe problems if you have not thoroughly tested them.

{{% /alert %}}

Making changes

{{% alert title="Note" color="danger" %}}

Always, always, ALWAYS perform thorough and complete testing for Workato changes.

{{% /alert %}}

Currently, changes to Workato are made in Workato itself.

SECTION: security/customer-support-operations/workflows/gitlab/_index.md

SECTION: security/customer-support-operations/workflows/gitlab/mirroring.md

Process

To setup mirroring in a secure and scalable way, use the following steps:

1. Login as the gl-support-bot user
1. Navigate to the source project
1. Add a new mirror
 - Git repository URL: ssh://gitlab.com/full/path/to/project.git
 - Mirror direction: Push
 - Authentication method: SSH public key
 - Username: git
 - Keep divergent refs: unchecked
 - Mirror branches: Mirror only protected branches
1. Copy SSH public key from new mirror
1. Navigate to the target project
1. Add a new deploy key (under Settings > Repository > Deploy Keys)

- Title: full/path/to/project
 - Key: The value copied in step 4
 - Grant write permissions to this key: checked
 - Expiration date (optional): leave it blank
1. Navigate to the source project
 1. Force an update on the mirror (under Settings > Repository > Mirroring repositories)

SECTION: security/customer-support-operations/workflows/gitlab/project-setup.md

{{% alert title="Note" color="danger" %}}

We all must follow this process to ensure our projects are setup properly in a secured and maintainable way. If every you are questioning what to do, please reach out to your fellow team members.

{{% /alert %}}

For all projects we create, we follow a specific process.

Step 1 - Use a project template

The starting place to setup a project for Customer Support Operations is going to be with one of our project templates. We currently have 5 to choose from:

- Zendesk Global App
- Zendesk US Government App
- Zendesk Sync Repo
- Forms
- General

The exact one you will use depends on what the project's purpose is. If you are ever unsure which to use, reach out to the other Customer Support Operations team members to determine which one is thhe ideal one ot use.

Knowing which template you will be using will then let you move to creating the project. To do this:

1. Navigate to the correct group
 - The correct group will be somewhere under the GitLab Support Readiness namespace.
 - If you are ever unsure which to use, reach out to the other Customer Support Operations team members to determine which one is the ideal one ot use
1. Click New project at the top-right of the page
 - If you do not see that button, please reach out to a Fullstack Engineer, Customer Support Operaitons for assistance.
1. Click Create from template
1. Click the Groups tab
1. Locate the project template you are to use and click Use template to the right of it
1. Enter a sensible project name

1. Ensure the Visibility Level is set to Private
1. Click Create project

This will then use the project template to create the project (this can take a few minutes to complete).

Step 2 - Verify settings

With the project created, we need to manually verify all the settings. While the project template should cover most of these, you should always check all settings to ensure they align. If anything differs from the below list, correct it.

- General
 - Visibility, project features, permissions
 - Project visibility: Private
 - Additional options
 - [x] Require authentication to view media files
 - Issues: disabled
 - Repository: enabled
 - Merge requests: enabled
 - Forks: disabled
 - Git Large File Storage (LFS): disabled
 - CI/CD: enabled
 - Container registry
 - Analytics: disabled
 - Requirements: disabled
 - Security and compliance: disabled
 - Wiki: disabled
 - Snippets: enabled
 - Package registry: disabled
 - Package registry: disabled
 - Model experiments: disabled
 - Pages: disabled
 - NOTE: This will be enabled if making a form type project
 - Monitor: disabled
 - Environments: disabled
 - Feature flags: disabled
 - Infrastructure: disabled
 - Releases: disabled
 - GitLab Duo: enabled
 - Email notifications
 - [x] Enable email notifications
 - [x] Include diff previews
 - [x] Show default emoji reactions
 - [x] Warn about Potentially Unwanted Characters
 - CI/CD Catalog project: disabled
- Badges
 - None
- Service Desk
 - Disabled

- Integrations
 - None
- Webhooks
 - None
 - NOTE: You might be adding some later if needed, but initially there should not be any
- Access tokens
 - None
 - NOTE: You might be adding some later if needed, but initially there should not be any
- Repository
 - Branch defaults
 - Default branch: master
 - ☒ Auto-close references issues on default branch
 - Branch rules
 - All branches
 - Squash commits: Require
 - master
 - Requires CODEOWNERS approval
 - Allowed to merge: Maintainers
 - Allowed to push and merge: 1 user
 - Push rules
 - ☐ Reject unverified users
 - ☐ Reject inconsistent user name
 - ☐ Reject unsigned commits
 - ☐ Reject commits that aren't DCO certified
 - ☐ Do not allow users to remove Git tags with git push
 - ☐ Check whether the commit author is a GitLab user
 - ☐ Prevent pushing secret files
 - Require expression in commit messages: blank
 - Reject expression in commit messages: blank
 - Branch name: blank
 - Commit author's email: blank
 - Prohibited file names: blank
 - Maximum file size (MB): 0
 - Mirroring repositories
 - None
 - NOTE: You might be adding some later if needed, but initially there should not be any
 - Protected branches
 - master
 - Allowed to merge: Maintainers
 - Allowed to push and merge: GitLab Support Bot
 - Allowed to force push: disabled
 - Code owner approval: enabled
 - Protected tags
 - None
 - Deploy tokens
 - None
 - Deploy keys
 - None
- Merge Requests
 - Merge requests

- Merge method: Merge commit
- Merge options
 - ☐ Enable merged results pipelines
 - NOTE: In rare situations this may be enabled, but never by default
 - ☐ Automatically resolve merge request diff threads when they become outdated
 - ☒ Show link to create or view a merge request when pushing from the command line
 - ☒ Enable "Delete source branch" option by default
- Squash commits when merging: Require
- Merge checks
 - ☒ Pipelines must succeed
 - NOTE: This will vary based on project template type.
 - ☒ All threads must be resolved
 - ☐ Status checks must succeed
- Status checks
 - None
- Merge suggestions: blank
- Merge commit message template

plaintext

Merge branch '%{sourcebranch}' into '%{targetbranch}'

%{title}

%{issues}

See merge request %{reference}

- Squash commit message template

plaintext

%{title}

- Default description template for merge requests: blank`
- Merge request approvals
 - Coverage-Check: disabled
 - Minimum required approvals: 0
 - Approval settings
 - ☒ Prevent approval by author
 - ☒ Prevent approvals by users who add commits
 - ☒ Prevent editing approval rules in merge requests
 - ☐ Require user re-authentication (password or SAML) to approve
 - When a commit is added: Removal all approvals
- Merge request branch workflow
 - None
- CI/CD
 - General pipelines
 - ☐ Public pipelines
 - ☒ Auto-cancel redundant pipelines

- ☒ Prevent outdated deployment jobs
 - ☒ Allow job retries even if the deployment job is outdated.
- ☒ Use separate caches for protected branches
- Minimum role required to cancel a pipeline or job: Developer
- CI/CD configuration file: blank
- Git strategy: git fetch
- Git shallow clone: 20
- Timeout: 1h
- Automatic pipeline cleanup: blank
- Auto DevOps
 - ☐ Default to Auto DevOps pipeline
- Protected environments
 - None
- Pipeline trigger tokens
 - None
 - NOTE: You might be adding some later if needed, but initially there should not be any
- Automatic deployment rollbacks
 - ☐ Enable automatic rollbacks
- Deploy freezes
 - None
- Job token permissions
 - Authorized groups and projects: Only this project and any groups and projects in the allowlist
- CI/CD job token allowlist
 - Only the the project itself should be present in the list
 - Limit access from this project (Deprecated): disabled
 - Add an existing project to the scope
 - Only the the project itself should be present in the list
- Secure files
 - None
- Pipeline subscriptions
 - Subscriptions
 - None
 - Subscribed to this project
 - None
- Monitor
 - Error tracking
 - Enable error tracking
 - ☐ Active
 - Error tracking backend: gitLab
- Alerts
 - None
- Incidents
 - Active: disabled
- Status page
 - ☐ Active
 - Status page URL: blank
 - S3 Bucket name: blank
 - AWS region: blank
 - AWS access key ID: blank

- AWS Secret access key: blank

Step 3 - Add the correct memberships

After ensuring the settings are accurate, you need to add the correct memberships to the project.

- If this is a US Government only project, then add the group gitlab-support-readiness/readiness-us-gov-only as Maintainers
- For everything else, then add the group gitlab-support-readiness/all-readiness as Maintainers

Step 4 - Setup CI/CD variables

With that done, you should get your CI/CD variables setup and in place. The parent namespace sets many of them for you, so the ones you would need to focus on are:

- API token values (Zendesk, GitLab, Calendly, etc.)
- Slack webhooks

Double check what variables are set by the group (listed on the page you would add variables to the project) to determine which variables you need to add.

Step 5 - Add your code

Here you will add your code. You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

Step 6 - Anything else

Here you will do whatever else might be needed for the project. This could include setting up webhooks, mirroring the code to a different project, etc.

SECTION: security/customer-support-operations/workflows/gitlab/support-super-form.md

{{% alert title="Note" color="primary" %}}

Support Super Form changes are classified as ad-hoc. Once changes are merged into the default branch, the form will be generated and live.

These also usually involve changes to other components we manage. Often the Support Super Form itself is the last item to be updated.

This can mean changes can cause severe problems if you have not thoroughly tested them.

{{% /alert %}}

Making changes

Making the changes would be done by modifying the files within the project. You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

Once the MR is merged, they will then be used to populate the form.

SECTION: security/customer-support-operations/workflows/gitlab/support-team-yaml-files.md

{{% alert title="Note" color="primary" %}}

Support Team YAML file changes are classified as ad-hoc. Once changes are merged into the default branch, they are synced to Redis and live.

This can mean changes can cause severe problems if you have not thoroughly tested them.

{{% /alert %}}

Request for changes

Requests to make changes to the Support Team YAML files require specific levels of approval.

For the following items, the target's manager must add their approval on the MR:

- Title
- Working Hours
- Focuses
- Zendesk Groups (either instance)
- GitLab worksaccountdeletion value
- Display name / Alias updates

The approval is required to proceed. If it has not been granted, do not proceed on the MR.

Making changes

{{% alert title="Note" color="primary" %}}

Customer Support Operations is not normally the one making the changes. We usually act in a "review+merge" capacity.

{{% /alert %}}

Making the changes would be done by modifying the files within the project. You should also do this in a way that creates a MR. Said MR should always be peer reviewed before merging (the MR should enforce this).

Once the MR is merged, they will then be synced into Redis (and used in our wider systems).

SECTION: security/customer-support-operations/workflows/gitlab/working-issues.md

{{% alert title="Note" color="primary" %}}

This does not apply for Incident type issues. Please see our incident workflow for more information on those.

{{% /alert %}}

Working Issues

Following Scheduling, issues will be scheduled into iterations and assigned out. So long as you're the assignee, you're the DRI for:

- Making progress through the stages
- Coordinating with others when you're blocked or unsure of what to do next
- Making weekly updates to issues assigned to you

Weekly updates

Every week, on your last work day (for example, Friday) review your assigned issues and:

- post a brief update
- update the health status of the issue.
- update the stage, if appropriate. For example, if the issue is "Blocked" make sure that's the current stage.

For convenience, there is a Customer Support Ops Team group comment template called "Issue Status Update" that has all of the components.

Generally, your comment should outline:

- The current progress you've made: include tasks completed, what you're working on, or even the % of the work you've completed.
- Your plan going forward: what next steps you'll take (and when), and when you estimate to finish.
- If there is any support you need, for example if you are blocked and need something from another team.

These will be reviewed each week and used to generate the required weekly slack updates for teams in Security.

Stages

The work to be done on an issue largely depends on the stage the issue is in. Please note the assignee on an issue will change frequently as it moves from stage to stage.

A standard progression for an issue, will look like this:

```
mermaid
graph TD;
  Type-->|Bug| BugTriage
  Type-->|Feature| FeatureTriage
  Type-->|Administrative| AdministrativeTriage
  BugTriage--> BugPlanning
  BugPlanning--> Development
  FeatureTriage--> FeaturePlanning
  FeaturePlanning--> FeatureScheduling
  FeatureScheduling--> IterationTime
  IterationTime-->|Yes| Development
  IterationTime-->|No| FeatureQueued
  FeatureQueued--> IterationTime
  Development--> Validation
  Validation-->|Rejected| Development
  Validation-->|Approved| Implementation
  AdministrativeTriage--> AdministrativeDevelopment
  AdministrativeDevelopment-->|Skip validation| Implementation
  Type{What type of issue?}
  IterationTime{Is it the current iteration?}
  BugTriage[Triage]
  BugPlanning[Planning]
```